--- Gemini Code Assist Log --- Timestamp: 2023-10-27THH:MM:SSZ (Actual timestamp will be inserted)

Gemini Operations Log

- Created new file: d:/ERP/project/src/frontend/common/main.css
  - Defined CSS for common UI components: AppHeader, AppFooter, AppPageTitle, AppGrid, AppTable, AppModal.
- Modified file: d:/ERP/project/README.md
  - Updated "Common UI Components" section to include descriptions for the newly defined components.

2026-04-07

- Read `d:\ERP\project\package.json` to understand the project tech stack (Vue 3, Vite, Pinia, Playwright, Vitest).
- Read `d:\ERP\project\README.md` to analyze the original boilerplate content.
- Updated `d:\ERP\project\README.md` to include a comprehensive design document, explaining the architectural strategy:
  - Domain-Driven Design separating the application into distinct modules (`FI`, `HR`, `MM`, `SD`, `SYS`, `TEMP`).
  - Outlined the frontend architecture using Views, Components, Stores, and API services for each module.
  - Outlined the backend MVC-like architecture utilizing Controllers, Services, Repositories, and Models.
- Created `d:\ERP\project\gemini.log` to track tasks performed by the Gemini CLI agent.
- Updated `d:\ERP\project\src\frontend\common\main.css` to define all design elements (colors, spacing, typography, layouts) dynamically using CSS Variables (`--app-*`), ensuring full compatibility for JS runtime integration.
- Created `d:\ERP\project\src\frontend\common\theme.js` to provide a JS interface (`setCssVariable`, `getCssVariable`, `applyTheme`) allowing programmatic control over CSS styling (e.g., Light/Dark mode).
- Updated `d:\ERP\project\src\frontend\main.js` to fix the `App.vue` relative import path and to import the global `main.css` file. [UI Update] 공통 UI 시스템 스타일 및 문서화 업데이트
- `main.css` 파일의 프로젝트 전역 변수를 활용하여 `.app-footer`, `.app-page-title`, `.app-grid`, `.app-table`, `.app-modal` 스타일을 추가로 정의함.
- `README.md`의 'Common UI Components' 섹션에 새롭게 정의한 스타일 클래스들의 사용법과 역할을 문서화함.

[Environment Setup] 환경 변수 확인 및 OAuth 설정 완료

- `src/.env` 파일에 Google 클라이언트 ID 및 API 키, DB 연결 정보가 설정된 것을 확인함.
- `TODO.md`의 Phase 1 요구사항 중 'Google OAuth Setup' 완료 처리함.

[Backend Setup] Phase 1 - Auth API 백엔드 기본 구조 및 엔드포인트 구현 (진행중)

- 백엔드 구동을 위한 `express`, `cors`, `cookie-parser`, `dotenv`, `google-auth-library`, `jsonwebtoken`, `mysql2`, `winston` 등의 패키지를 설치함. (일부 권한 오류로 재설치 필요함 확인)

- `src/backend/server.js` 생성: Express 서버 기본 설정 및 `/api/health`, `/api/auth` 라우터 연결
- `src/backend/module_sys/routes/authRoutes.js` 생성: 구글 로그인 및 로그아웃 라우팅
- `src/backend/module_sys/controllers/authController.js` 생성: `google-auth-library`로 구글 토큰 검증, 유저 생성/조회, JWT 발급 및 쿠키 설정 로직 구현
- `src/backend/module_sys/services/userService.js` 생성: DB에 접근해 유저가 없으면 새로 등록하고 반환하는 로직 구현
- `src/backend/common/config/db.js`, `logger.js` 파일을 Node.js ES 모듈 (`import/export`) 형식에 맞게 수정함.

[Backend Bugfix & Verification] 실패 지점 복구 및 서버 구동 테스트 완료

- 패키지 충돌 및 권한 문제 우회를 위해 `google-auth-library`, `jsonwebtoken`, `mysql2` 등 필수 의존성 재설치 완료.
- `server.js`와 `db.js` 내의 `.env` 파일 경로가 실제 위치(`src/.env`)와 다르게 설정되어 있던 이슈를 수정함(`../../.env` -> `../.env` 등).
- 노드 백엔드 서버(`node src/backend/server.js`) 정상 구동 확인.
- `http://localhost:3000/api/health` 헬스 체크 API 호출을 통해 정상 작동 검증 성공.
- `TODO.md`에서 'Backend Auth API' 및 'Token Management' 구현 항목 완료 처리.

[Frontend Update] 메인 페이지(Dashboard) 및 레이아웃 구현 완료

- `src/frontend/common/layouts/DefaultLayout.vue` 생성: 상단 `AppHeader`, 좌측 `app-sidebar`(모듈 네비게이션), 중앙 `app-content`, 하단 `AppFooter`로 구성된 표준 ERP 레이아웃 구현.
- `src/frontend/views/HomeView.vue` 생성: `AppGrid`, `AppCard`, `AppTable` 등 앞서 만든 공통 UI 컴포넌트들을 조합하여 시스템 개요 및 최근 활동 내역이 표시되는 메인 대시보드 화면 구현.
- `src/frontend/router/index.js` 수정: 최상위 경로(`/`) 접근 시 `DefaultLayout` 내부에 `HomeView`가 렌더링되도록 라우팅 구조 개편.
- `src/frontend/App.vue` 수정: 기본 보일러플레이트 코드를 제거하고 `<RouterView />`로 대체하여 Vue Router가 제어하도록 변경함.

[Documentation] 시스템 환경 변수(.env) 명세 추가 및 하드코딩된 값 변수화

- 백엔드 CORS 오리진 주소(`http://localhost:5173`)를 `process.env.CORS_ORIGIN`으로 변수화함 (`src/backend/server.js`).
- JWT 만료 시간(`12h`) 및 쿠키 유지 시간(`43200000`)을 각각 `process.env.JWT_EXPIRES_IN`, `process.env.JWT_COOKIE_MAX_AGE`로 변수화함 (`src/backend/module_sys/controllers/authController.js`).
- 향후 사용할 프론트엔드 API 엔드포인트(`VITE_API_BASE_URL`)를 포함해 새롭게 추가된 변수들의 명세를 `README.md Environment Variables (.env)` 섹션에 업데이트함.
- `TODO.md`의 'Ongoing Maintenance' 항목에 환경 변수 명세 관리에 대한 태스크를 추가함.

[Documentation & Architecture] 개발자 참조용 전역 상수(Constants) 파일 정의

- 하드코딩 방지 및 시스템 일관성 유지를 위해 읽기 전용(`Object.freeze()`) 전역 상수 파일을 선언함.
- **Frontend**: `src/frontend/common/constants.js` 생성 (모듈명, 권한, 날짜 포맷, 페이징 기본값, API 타임아웃 등 정의).
- **Backend**: `src/backend/common/config/constants.js` 생성 (HTTP 상태 코드, 권한, 페이징 기본값, DB 기본값 등 정의).

- `README.md`에 `Global System Constants (Read-Only)` 섹션을 추가하여 개발자가 상수들을 어떻게 호출하고 사용해야 하는지 명세함.
- `TODO.md`에 지속적인 상수 관리를 위한 유지보수 태스크 추가함.

[Documentation & Architecture] 데이터 포맷팅용 전역 유틸리티(Formatters) 함수 정의

- 모듈 간 일관된 데이터 출력 및 변환을 위해 전역 포맷터 함수를 시스템 유틸리티로 구성함.
- **Frontend**: `src/frontend/common/utils/formatters.js` 생성 (`formatDate`, `formatDateTime`, `formatCurrency`, `maskData` 등 화면 출력용 함수 정의).
- **Backend**: `src/backend/common/utils/formatters.js` 생성 (`formatDateForDB`, `formatDateTimeForDB`, `parseBoolean` 등 DB 삽입 및 데이터 검증용 함수 정의).
- `README.md`에 `Global System Formatters (Utility Functions)` 섹션을 신설하여 각 포맷터 함수의 기능과 호출 방식을 문서화함.
- `TODO.md` 유지보수 항목에 전역 유틸리티/상수 관리를 통합하여 업데이트함.

[Backend Setup] Phase 1 - 인증 미들웨어 및 User 모델 고도화

- `src/backend/common/middleware/authMiddleware.js`: 기존 CommonJS를 ES 모듈로 변환하고, JWT 토큰을 쿠키(또는 헤더)에서 읽어 검증하도록 로직을 완전 재작성함. (만료 여부 및 조작 방지)
- `src/backend/module_sys/models/user.js`: 사용하지 않는 Sequelize 보일러플레이트를 제거하고, 현재 사용 중인 Raw Query에 맞는 데이터 스키마 명세(`UserModel`) 객체로 교체함.

[Frontend Setup] Phase 4 - Google 로그인 프론트엔드 UI 연동

- `src/frontend/common/layouts/AuthLayout.vue` 및 `src/frontend/views/LoginView.vue` 생성: 구글 로그인 전용 레이아웃 및 뷰를 구현함. 구글 Identity Services의 'Sign in 정 연동.
- `index.html`에 `https://accounts.google.com/gsi/client` 스크립트를 추가하여 구글 로그인 SDK를 로드함.
- `src/frontend/views/LoginView.vue` 수정: 비동기로 로드되는 구글 SDK(`window.google` 객체)가 초기화되기 전에 렌더링을 시도하여 버튼이 보이지 않던 이슈를 발견하고, `setInterval`을 이용한 폴링(Polling) 방식으로 SDK 로드 직후 버튼을 렌더링하도록 안전 장치를 추가함.
- `src/frontend/router/index.js`: `/login` 라우트를 추가하고, `beforeEach` 가드를 통해 인증되지 않은 사용자가 대시보드(`/`)에 접근 시 `/login`으로 리다이렉트되도록 구현함.
- `vite.config.js`: 프론트엔드 API 호출(`.fetch('/api/...')`)이 백엔드 서버(`localhost:3000`)로 올바르게 넘어가도록 `proxy` 설정을 추가함. (환경 변수 연동 포함)
- `src/frontend/common/layouts/DefaultLayout.vue`: 우측 상단에 현재 로그인한 사용자의 이름을 표시하고, 로그아웃 버튼을 통한 JWT 쿠키/스토리지 삭제 로직을 연결함.
- `TODO.md`에서 Phase 1 및 Phase 4의 모든 항목 완료 처리.

[Database Verification] 로그인 테스트를 위한 sysUser 테이블 점검

- 백엔드 서버 및 Vite 개발 서버 구동 상태를 확인하고, 데이터베이스(`study`) 내에 로그인한 유저 정보를 담을 `sysUser` 테이블이 정상적으로 존재하는지 확인 및 자동 생성 스크립트 구동을 완료함.

[Bugfix] 프론트엔드 환경 변수 로드 문제 해결

- 프론트엔드의 `VITE_GOOGLE_CLIENT_ID` 값이 `undefined`로 떨어져 구글 로그인 버튼이 렌더링되지 않던 문제를 확인함.
- 기본적으로 프로젝트 루트에서 `.env`를 찾는 Vite 설정 파일(`vite.config.js`)에 `envDir: './src'` 옵션을 추가하여, 클라이언트 사이드 코드(`import.meta.env`)가 `src/` 폴더 내의 환경 변수를 정상적으로

읽어올 수 있도록 수정함.
- 수정 사항 적용을 위해 백그라운드 노드 프로세스들을 모두 정리하고 개발 서버 재기동을 완료함.

[Architecture & Planning] 시스템 모듈(SYS) 프론트엔드 뷰 구현 계획 수립

- 백엔드의 시스템 모듈(`module_sys`) 모델(사용자, 권한, 메뉴, 공통코드, 시스템 설정 등)과 1:1로 매칭되는 프론트엔드 관리 화면들을 먼저 구현하기로 방향성을 확립함.
- `TODO.md`에 'Phase 6: System Module (SYS) UI Implementation' 항목을 새롭게 추가하여 구현 목표를 명확히 함.

2026-04-08

[Architecture & Planning] 시스템 모듈(SYS) 프론트엔드 화면 네이밍 룰 및 매핑 재정립

- `src/frontend/module_sys/README.md` 작성: SYS 모듈 하위 폴더 분리(SYST01~SYST05) 및 목록(V001)/상세(V002) 분리 네이밍 룰 확립.
- `TODO.md` 업데이트: 새 네이밍 룰에 맞춰 Phase 6 및 Phase 7 구현 태스크를 구체적으로 목록화하고 분할함.
- `gemini.md`에 지금까지의 모든 작업 내역과 앞으로의 규칙 변경 사항을 기록함.

[Refactoring] 시스템 모듈(SYS) 프론트엔드 화면 재배치 및 라우터 수정

- `src/frontend/module_sys/views` 에 있던 기존 화면 파일들을 명명 규칙에 따라 각각의 하위 폴더(SYST00 ~ SYST05) 로 분리 및 이름 변경 완료 (예: `UserManagementView.vue` -> `SYST01/SYST01V001.vue`).
- 사용하지 않는 기존 `views` 폴더 삭제.
- `src/frontend/router/index.js` 내의 라우팅 컴포넌트 경로들을 새로운 폴더 및 파일명에 맞게 일괄 변경 적용 완료.

[Architecture & Planning] 시스템 모듈(SYS) 테이블 명세서 관리(SYST06) 설계 추가

- `src/frontend/module_sys/README.md` 업데이트: 테이블/컬럼/인덱스의 DDL을 관리, 추적, 생성하는 `SYST06`(테이블 명세서 관리) 영역을 추가하고 `SYST06V001`(조회), `SYST06V002`(상세, 이력, SQL 생성) 화면 매핑 정보를 명세함.
- `TODO.md` 업데이트: Phase 6 항목에 `SYST06 Table Specifications` 관련 태스크를 추가하고, 데이터베이스 변경 로그(`sysTableHistory`) 기반의 SQL DDL 생성(Generate SQL) 요구사항을 기록함.

2026-04-09

[Backend Setup] Phase 8 - SYST06 테이블 명세서 관리 모델, 서비스, 컨트롤러 구현

- `src/backend/module_sys/models/tableSpec.js`: 제공된 DDL 기반으로 `sysTable`, `sysFields`, `sysTableHistory` 등 테이블들의 Raw Query 스키마 명세 및 Mock Data 정의.
- `src/backend/module_sys/services/tableSpecService.js` 및 `tableSpecController.js`: 테이블 목록 및 상세 조회(필드, 인덱스, 이력 조인) 비즈니스 로직과 API 컨트롤러 구현. (Mock Data Fallback 적용)

[Frontend Setup] Phase 8 - SYST06 테이블 명세서 관리 화면 구현

- `src/frontend/module_sys/views/SYST06/SYST06V001.vue`: 테이블 명세서 목록 화면 구현 (검색, 페이징, AppTable 연동).

- `src/frontend/module_sys/views/SYST06/SYST06V002.vue`: 테이블 명세서 상세 조회 화면 구현 (Master-Detail 구조, Fields/Indexes/History 탭 UI).

[Frontend & Routing] Phase 8 - 라우터 매핑 및 SQL DDL 생성 UI 구현

- `src/frontend/router/index.js`: `SYST06V001` (목록) 및 `SYST06V002` (상세) 화면을 `/sys/tables` 및 `/sys/tables/:tablen` 경로에 맵핑 완료.
- `src/frontend/module_sys/components/SqlExecutionModal.vue` 생성: 테이블 상세 화면에서 "SQL DDL 생성" 버튼 클릭 시, 백엔드의 `generate-sql` API를 호출하여 모달 팝업으로 DDL을 출력하는 UI 컴포넌트 구현.
- `TODO.md` 업데이트: `Frontend: SQL Execution UI` 항목 완료 처리.

[Bugfix & Refactoring] 시스템 안정성 향상 및 라우터 오류 수정

- `src/backend/module_sys/services/tableSpecService.js`: SQL DDL 생성 로직(`generateSql`) 내의 템플릿 리터럴 문법(SyntaxError) 오류 수정 (백틱 및 줄바꿈 이스케이프 정상화).
- `src/backend/module_sys/controllers/authController.js`: 구글 로그인 응답 시 DB 스키마에 없는 필드 참조로 인한 `undefined` 오류를 방지하기 위해 인증 페이로드의 데이터를 반환하도록 수정. 안정성을 위해 `node:process` 모듈 명시적 임포트 추가.
- `src/frontend/router/index.js`: 폴더 네이밍 룰 개편(`SYST00`~`SYST05`)에 맞춰 라우터 컴포넌트의 임포트 경로를 수정하여 Vite 모듈 분석(import-analysis) 에러 해결.

[Bugfix & Recovery] SYST06 화면 및 모달 파일 위치 오배치 복구

- `src/frontend/module_sys/components/SqlExecutionModal.vue`: `router` 폴더에 잘못 생성되었던 SQL 실행 모달 파일을 올바른 `components` 폴더 위치로 복구.
- `src/frontend/module_sys/SYST06/SYST06V001.vue`, `SYST06V002.vue`: 임시 템플릿 생성 과정에서 덮어씌워졌던 테이블 명세서 목록 및 상세 뷰 코드를 원래 기획 명세에 맞는 마스터-디테일 폼 그리드 구조로 전면 복구함.

[Frontend Implementation] SYST06 및 SYST00 프론트엔드 뷰 구현

- `src/frontend/module_sys/SYST06/SYST06V001.vue` 생성: 테이블 명세서 관리 목록 화면, `sysTable` 검색 및 페이징, AppTable 연동 등 구현.
- `src/frontend/module_sys/SYST06/SYST06V002.vue` 생성: 테이블 명세서 상세 조회 및 수정 화면, 마스터(기본 정보)-디테일(필드/인덱스/변경 이력 탭) 구조 구현.
- `src/frontend/module_sys/components/SqlExecutionModal.vue` 생성: `SYST06V002`에서 호출하는 SQL 스크립트 실행 모달 컴포넌트 구현.
- `src/frontend/module_sys/SYST00/SYST00V001.vue` 생성: 시스템 상태, 최근 활동 및 접속 통계 등을 확인할 수 있는 메인 대시보드 화면 구현.
- `TODO.md`에서 'SYST00 System Dashboard' 구현 항목 완료 처리.

2026-04-10

[Backend & Frontend Setup] 권한 기반 동적 메뉴 시스템 구현

- 사용자의 권한(`sysUserRole`, `sysRoleMenu`)에 따라 허용된 메뉴만 조회하여 반환하는 백엔드 API 구현.
- `src/backend/module_sys/services/menuService.js` 생성: `sysUserRole`, `sysRoleMenu`, `sysMenu` 조인 쿼리 및 DB 데이터 부재 시 사용할 데이터 폴백(Mock Data) 구현.

- src/backend/module_sys/controllers/menuController.js 및 menuRoutes.js 생성, server.js에 /api/sys/menus 라우트 등록.
- src/frontend/common/components/AppSidebar.vue 생성: 기존 하드코딩된 aside 메뉴를 대체하는 동적 사이드바 컴포넌트 개발 (main.css 스타일 호환 유지).
- src/frontend/common/layouts/DefaultLayout.vue: AppSidebar 컴포넌트를 연동하여 권한에 맞는 메뉴만 렌더링되도록 수정.

[Frontend Update] 3레벨 계층 구조 메뉴 및 동적 펼침 기능 구현

- src/backend/module_sys/services/menuService.js: Mock Data에 3레벨 계층 구조 테스트를 위한 데이터 추가 (parentMenuId 명시).
- src/frontend/common/components/AppSidebar.vue: 백엔드에서 받아온 평면적인 메뉴 목록을 parentMenuId를 기반으로 트리(Tree) 구조로 재귀적으로 조립하는 buildTree 함수 구현.
- 3레벨까지 지원되는 중첩 v-for 문과 스타일(padding-left)을 추가하여 부모-자식-손자 메뉴가 깔끔하게 렌더링되도록 뷰(View) 개선.
- 현재 접속 중인 라우트(route.path)를 감지하여 해당하는 부모 메뉴가 자동으로 펼쳐지도록(Open) 처리하는 로직(openActiveMenus) 추가.
- 사용자의 클릭 없이 마우스 호버(mouseenter / mouseleave) 만으로도 하위 메뉴가 자연스럽게 펼쳐지고 닫히도록 Vue 반응형 상태(isHovered)를 활용해 개선.
- DB에 parentMenuId가 NULL 대신 빈 문자열('')로 입력되어 있을 경우 계층 구조가 평탄화(Flatten)되어 버리는 오류를 방지하기 위한 예외 처리(.trim() !== '') 추가.

[Backend & Frontend Setup] Phase 6 & 7 - SYST03 메뉴 관리 (Menu Management) CRUD 화면 및 API 구현

- src/backend/module_sys/services/menuService.js: sysMenu 테이블에 대한 getAllMenus(검색, 페이징 포함), getMenuById, createMenu, updateMenu, deleteMenu 함수 구현 및 데이터 부재 시 Mock 데이터 폴백 로직 통합.
- src/backend/module_sys/controllers/menuController.js: CRUD 비즈니스 로직을 연결하는 라우터 핸들러 함수들 추가.
- src/backend/module_sys/routes/menuRoutes.js: GET /, POST /, GET /:id, PUT /:id, DELETE /:id 등 RESTful API 엔드포인트 정의 및 verifyToken 미들웨어 적용.
- src/frontend/module_sys/SYST03/SYST03V001.vue: 시스템 전체 메뉴를 조회하는 목록 화면 구현. AppPagination과 연동된 서버 사이드 페이징 및 ID/메뉴명 기반 검색 기능 구현 (Phase 7 요구사항 준수).
- src/frontend/module_sys/SYST03/SYST03V002.vue: 개별 메뉴 상세 조회, 수정, 신규 등록 및 삭제가 가능한 폼(Form) 화면 구현. isNew 라우트 파라미터('new')를 활용하여 등록/수정 모드 동적 전환.
- src/frontend/router/index.js: /sys/menus/:id 라우트를 추가하여 상세 화면으로의 네비게이션 연결 구성 완료.
- TODO.md에서 Phase 6의 'SYST03 Menu Management' 항목과 Phase 7의 'SYST03V001' 페이징 적용 항목 완료 처리.

[Backend & Frontend Setup] Phase 6 & 7 - SYST05 시스템 설정 (System Settings) CRUD 화면 및 API 구현

- src/backend/module_sys/services/configService.js: sysConfig 테이블에 대한 getAllConfigs(검색, 페이징 포함), getConfigById, createConfig, updateConfig, deleteConfig 함수 구현 및 데이터 부재 시 Mock 데이터 폴백 로직 추가 (SYS_LANG, THEME 등 기본 설정 세트).
- src/backend/module_sys/controllers/configController.js 및 configRoutes.js: /api/sys/configs 엔드포인트를 통한 REST API 구축. server.js에 라우트 등록.

- `src/frontend/module_sys/SYST05/SYST05V001.vue`: 시스템 설정 목록 조회, 페이징 (`AppPagination`) 및 검색 기능 연동 (Phase 7).
- `src/frontend/module_sys/SYST05/SYST05V002.vue`: 시스템 설정 개별 상세 조회, 신규 등록 및 수정, 삭제를 위한 폼(Form) 컴포넌트 개발.
- `src/frontend/router/index.js`: `/sys/settings/:id` 경로 추가 매핑.

2026-04-11

[Bugfix] 시스템 관리 모듈 (SYS) 렌더링 오류 및 라우터 경고 수정

- `src/frontend/common/components/AppTable.vue`: `<AppTable>` 컴포넌트가 `columns`와 `data` Props를 필수로 받도록 설계되었으나, 테이블 태그를 직접 자식으로 넣으면서 발생하던 렌더링 크래시 오류 해결. 컴포넌트에 `@row-click` 이벤트를 추가하여 마우스 포인터 스타일과 클릭 액션을 지원하도록 개선.
- `src/frontend/module_sys/SYST03/SYST03V001.vue`, `SYST05/SYST05V001.vue`: `<AppTable>` 사용 방식을 하드코딩된 HTML 태그에서 `tableColumns`와 `data` Props를 전달하는 방식으로 수정하고 데이터 바인딩 연동 처리 완료.
- `src/frontend/router/index.js`: Vue Router 4 버전부터 `next()` 콜백 사용이 Deprecated(더 이상 사용되지 않음) 됨에 따라, `beforeEach` 전역 네비게이션 가드에서 `next('/login')` 대신 `return '/login'` 형식으로 리다이렉트 경로를 반환하도록 수정하여 콘솔 경고 해결.

[Architecture & Refactoring] 프론트엔드 API 호출 모듈화 및 구조 개선

- 컴포넌트 내부에 하드코딩되어 있던 `fetch()` API 호출을 역할과 도메인(모듈)에 맞게 별도의 API 서비스 파일로 분리.
- `src/frontend/module_sys/api/menuApi.js` 생성: `axios` 인스턴스를 활용하여 `/api/sys/menus` 엔드포인트와 통신하는 `getMenus`, `getMenuDetail`, `saveMenu` 등의 함수 구현.
- `src/frontend/module_sys/api/configApi.js` 생성: `/api/sys/configs` 엔드포인트와 통신하는 함수 구현.
- `src/frontend/module_sys/SYST03/SYST03V001.vue`, `SYST05/SYST05V001.vue`: 기존의 `fetch` 로직을 제거하고, 새로 생성한 `menuApi.js`, `configApi.js`의 모듈화된 함수들을 임포트하여 사용하도록 리팩토링.
- `src/frontend/common/main.css`: 트리(Tree) 형태의 데이터를 표현하기 위한 `.app-tree-table` 전용 CSS 스타일 클래스 추가.

[Backend & Frontend Setup] Phase 6 - SYST02 권한/역할 관리 화면 및 매핑 구현

- `src/backend/module_sys/services/roleService.js`, `roleController.js`, `roleRoutes.js` 추가: `sysRole` 테이블 기본 CRUD뿐만 아니라 `sysRoleMenu`(권한별 메뉴), `sysUserRole`(사용자별 권한) 테이블 데이터를 조회하고 삭제/삽입(갱신)하는 통합 비즈니스 로직과 트랜잭션을 구현. `server.js`에 `/api/sys/roles` 마운트.
- `src/frontend/module_sys/api/roleApi.js` 작성: 생성된 서버 역할을 처리할 프론트 API 모듈 구현 (`getRoleMenus`, `updateRoleUsers` 등 포함).
- `src/frontend/module_sys/SYST02/SYST02V001.vue`: 기존 Mock UI를 서버 데이터 연동 기반의 `sysRole` 목록 페이지로 전면 개편(페이징, 검색).
- `src/frontend/module_sys/SYST02/SYST02V002.vue`: `sysRole` 마스터 정보(역할 ID, 설명 등) 등록/수정과 함께, 탭(Tab) 기반으로 1) **메뉴 권한(Role Menus)** 다중 체크박스 할당 2) **사용자 할당(Role Users)**을 1:N으로 맵핑 및 저장할 수 있는 Master-Detail 복합 폼 구현.
- `src/frontend/router/index.js`: 권한 상세 화면으로 이어지는 `/sys/roles/:id` 라우팅 추가.

[Backend Bugfix] SYST02 권한 관리 - 테이블 스키마(DDL)에 맞춘 로직 전면 수정

- `sysRole` DDL에 `roleNm` 컬럼이 없는 것을 반영하여 `roleService.js` 내 조회/저장 로직을 `description` 기준으로 완벽히 호환되도록 수정함.
- `sysRoleMenu` (권한별 메뉴) 및 `sysUserRole` (사용자별 권한) 매핑 처리를 위한 트랜잭션 기반 bulk 삽입/삭제 로직(`updateRoleMenus`, `updateRoleUsers`)을 `roleService`에 구현함.
- `roleController.js` 및 `roleRoutes.js`에 `/api/sys/roles/:id/menus` 및 `/api/sys/roles/:id/users` 등 N:M 관계의 라우트를 추가하고 전체 엔드포인트에 `verifyToken` 미들웨어를 적용함.

[Architecture Refactoring] 백엔드 Service와 Repository 계층 분리 및 Role 비즈니스 로직 수정

- `src/backend/module_sys/repositories/roleRepository.js` 신규 생성: 기존 `roleService`에 혼재되어 있던 데이터베이스 접근 로직(SQL 쿼리, 트랜잭션 등)을 전담하도록 분리함.
- `src/backend/module_sys/services/roleService.js` 리팩토링: `roleRepository`를 의존성으로 주입받아 비즈니스 로직(Mock 데이터 Fallback, 데이터 검증)에 집중하도록 개선함.
- **조회 로직 개선**: 기존에는 검색 조건(`search`)이 있을 때 Mock 데이터를 조회하지 못하는 버그가 있었으나, DB 전체 카운트를 확인하여 완전히 비어있을 경우 Mock 데이터 내에서 검색 및 페이징을 처리하도록 수정함.
- **등록 로직 개선**: 역할(Role) 등록 시 `roleId` 중복 여부를 사전에 Repository를 통해 확인하고, 이미 존재하는 경우 명확한 에러 메시지를 반환하여 DB 무결성 예외 처리를 비즈니스 로직 단에서 대응하도록 수정함.

[Bugfix & Refactoring] Role 모달의 API 메서드 불일치 및 구형 Sequelize 코드 제거

- `src/frontend/module_sys/api/roleApi.js`: 역할의 메뉴/사용자 할당 정보를 저장하는 API 호출을 백엔드 라우터에 맞게 `POST`에서 `PUT`으로 수정하여 404 오류 해결.
- `src/backend/module_sys/models/role.js`, `userRole.js`, `roleMenu.js`: 기존 CommonJS 방식의 Sequelize 보일러플레이트 코드를 제거하고, 프로젝트 규격(Raw Query)에 맞는 ES 모듈 스키마 객체 명세로 모두 치환 완료.

[Architecture Refactoring] 시스템 전면 Sequelize ORM 기반 MVC 모델 도입 및 고도화

- `src/backend/common/config/sequelize.js` 신규 생성: 기존 `db.js`(Raw Query Pool) 대신 순수 Sequelize ORM 전용 연결 설정 구성.
- `src/backend/module_sys/models/*` 전면 개편: 16개 이상의 엔티티 파일(`user.js`, `table.js`, `field.js`, `menu.js`, `role.js`, `codeHead.js` 등)을 ES6 Class 기반의 순수 Sequelize Model로 완전 전환.
- `src/backend/module_sys/models/index.js` 신규 생성: 모든 모델을 임포트하고 엔티티 간의 복잡한 관계(Associations - `hasMany`, `belongsTo` 등)를 중앙에서 선언하여 관리.
- `src/backend/module_sys/models/tableHistory.js` 신규 생성: 테이블 변경 이력을 관리하기 위한 전용 ORM 모델 추가 및 연동.
- `src/backend/module_sys/services/userService.js`, `tableSpecService.js` 전면 리팩토링: 서비스 내부에 섞여 있던 모든 원시 SQL 문자열(`sequelize.query`)을 제거하고, `findOne`, `findAll`, `findAndCountAll`, `create`, `update`, `destroy` 등 순수 ORM 객체 메서드와 트랜잭션(`t`)을 활용하는 객체지향적 코드로 완벽히 전환 완료.

[Architecture Refactoring] Role 및 Menu 서비스의 순수 ORM 적용

- `src/backend/module_sys/services/roleService.js`: 기존 `roleRepository` 의존성을 제거하고, `Role`, `RoleMenu`, `UserRole` 등의 Sequelize 모델 객체를 직접 사용하여 CRUD 및 트랜잭션 로직을 간결

하게 리팩토링함.
- `src/backend/module_sys/repositories/roleRepository.js`: 더 이상 사용되지 않아 내용 삭제 (Deprecated).
- `src/backend/module_sys/services/menuService.js`: `sysMenu` CRUD 로직을 ORM 기반으로 리팩토링 완료.

[Bugfix] 사이드바 메뉴 렌더링 누락 및 권한 폴백 로직 추가

- `src/backend/module_sys/services/menuService.js`: ORM 전환 과정에서 누락되었던 `getUserMenus` 함수를 복구함.
- `Menu`, `UserRole`, `RoleMenu` 모델을 조인하여 사용자의 권한에 맞는 메뉴만 반환하도록 구현함.
- **안전장치(Fallback) 추가**: DB에 메뉴가 아예 없거나, 사용자-권한 맵핑이 단 하나도 이루어지지 않은 초기 환경에서는 Mock 데이터나 전체 메뉴를 반환하도록 하여 UI가 멈추거나 잠기는 현상을 방지함.

[Refactoring] 라우터 경로 및 메뉴 데이터 구조 일치화

- `src/frontend/router/index.js`: 기획 구조 변경에 따라 라우팅 경로를 `/sys/users` 등에서 모듈 번호 기반인 `/sys/syst01` 등으로 전면 수정.
- `src/backend/module_sys/services/menuService.js`: 변경된 라우팅 구조가 사이드바 네비게이션에 반영되도록 Mock 데이터의 메뉴 ID와 `path` 속성을 `SYST00` ~ `SYST06` 규칙으로 매핑.
- `SYST02V001.vue`, `SYST02V002.vue`, `SYST03V001.vue`: 화면 내에서 동적으로 동작하는 `router.push()`의 경로를 `/sys/systXX` 로 동기화하여 상세 화면 진입 및 목록 복귀 시 404 에러 방지.

[Architecture Refactoring] 시스템 설정(Config) 서비스 순수 ORM 적용

- `src/backend/module_sys/models/config.js` 신규 생성: `sysConfig` 테이블에 대한 Sequelize 모델 정의.
- `src/backend/module_sys/models/index.js`: `Config` 모델 연동 및 내보내기 추가.
- `src/backend/module_sys/services/configService.js`: 원시 SQL(`pool.query`)을 전면 제거하고, `Config` 객체 모델을 활용한 CRUD, 페이징, 데이터 폴백 로직으로 완벽하게 전환 완료.